Matemática para Aprendizaje Automático

Clase 1 — Introducción al curso

Pablo A. Iturralde · Universidad Católica del Uruguay · 2026

Parte I

Un problema para pensar

Gatos y perros

Tenemos **10.000 imágenes** de gatos y perros. Queremos construir un programa que, dada una imagen nueva, diga si es gato o perro. **¿Cómo lo harían?**

Gatos y perros — el problema

Enfoque clásico:

Escribimos reglas a mano, por ejemplo: *si tiene orejas puntiagudas y bigotes, es gato*

Problema:

Las reglas son frágiles — ¿gatos con orejas caídas? ¿perros con bigotes?
La variabilidad es enorme. No escala.

La alternativa: que el programa *aprenda* las reglas a partir de ver mucho ejemplos.
Eso es **aprendizaje automático**.

Pero… ¿cómo se *aprende* de datos?

Para eso necesitamos traducir el problema a matemática:

1. **Representar** cada imagen como un objeto matemático (p. ej. un vector)
2. **Definir** un modelo con parámetros ajustables que nos de la salida ($\hat{y} = f(x; \theta)$)
3. **Medir** qué tan bien/mal le va al modelo (función de pérdida/*loss*: $L(\hat{y} - y)$)
4. **Minimizar** ese error ajustando los parámetros del modelo (optimización: $\min_\theta L(y - f(x; \theta))$)

Cada uno de estos pasos usa herramientas matemáticas concretas.

Eso es lo que vamos a aprender en este curso.

Parte II

El curso

Quién soy

- Profesor del Departamento de Ingeniería, UCU
- Investigador en aprendizaje automático y bioingeniería
- Ing. Eléctrica, Maestría en procesamiento de señales / ML
- Doctorado en Bioingeniería
- Co-creador de esta carrera y de este curso

> *"El aprendizaje resulta de lo que el estudiante hace y piensa, y solo de lo que el estudiante hace y piensa. El profesor puede promover el aprendizaje únicamente influyendo en lo que el estudiante hace para aprender."*

— Herbert Simon

Qué espero de ustedes

- **Tiempo**: dedicación sostenida, no solo antes de los exámenes
- **Esfuerzo**: persistir ante la confusión — ahí es donde se aprende
- **Práctica**: resolver problemas con lápiz, papel y código
- **Reflexión**: no solo hacer, sino entender *por qué* funciona

La matemática no se aprende leyendo. Se aprende haciendo.

Resumen del curso

Este curso profundiza la formación matemática para entender e implementar algoritmos de aprendizaje automático.

Cubrimos: álgebra lineal, factorizaciones matriciales, cálculo multivariado, optimización.

No cubrimos: probabilidad ni estadística (curso separado).

Filosofía: práctico y orientado a aplicaciones. No teoría abstracta.
Mucha programación, muchos ejercicios, mucho hacer.

Objetivos de aprendizaje

Al finalizar el curso, serán capaces de:

- Comprender la geometría de los espacios vectoriales y usarla para interpretar problemas de ML (PCA, regresión)
- Elegir y aplicar factorizaciones matriciales (LU, QR, SVD) según el problema
- Calcular gradientes, jacobianos y hessianos, y aplicar la regla de la cadena en grafos computacionales
- Formular problemas de optimización y resolverlos con descenso del gradiente y Lagrange
- Implementar en Python los métodos numéricos estudiados

Conocimientos previos requeridos

- **Álgebra lineal básica**: vectores, matrices, operaciones, sistemas de ecuaciones
- **Cálculo en una variable**: derivadas, integrales, Taylor
- **Programación en Python**: sintaxis básica, funciones, estructuras de datos

No se requiere cálculo multivariado, probabilidad ni estadística.

Equipo docente

- **Profesor:** Dr. Ing. Pablo Iturralde — pablo.iturralde@ucu.edu.uy
- **Asistente:** Juan Paroli — j.paroli@ucu.edu.uy
- **Asistente:** Mercedes Barrutia — m.barrutia@ucu.edu.uy

Horarios

Clases sincrónicas:

Martes y Jueves, 8:00 – 11:00

Consultas: a coordinar

Temario: cuatro unidades

Unidad	Tema	Semanas
U1	Álgebra Lineal — Geometría y Cómputo	5
U2	Sistemas Lineales y SVD	4
U3	Cálculo Multivariado y Backpropagation	4
U4	Optimización	3

Total: **16 semanas** (incluye exámenes y Semana Santa)

Evaluación

Componente	Peso	Modalidad
Pruebas escritas	60%	Individual, al final de cada unidad
Quizzes semanales	20%	Asincrónico, intentos ilimitados, 100% requerido
Laboratorios	10%	Notebooks en Python, se evalúa entrega
Participación	10%	Presencia activa, hojas de ejercicios

Aprobación

- Se requiere aprobar las **cuatro unidades** (60% en cada una)
- La última prueba escrita incluye preguntas de todas las unidades y sirve como **remediación** para una unidad reprobada
- Más de una unidad reprobada: **recursar**
- Nota final: promedio ponderado de las cuatro unidades (25% cada una)

Política de entregas

- **5 días de gracia** totales en el semestre (sumados entre todas las entregas)
- Después: **-33% por día** de demora
- Aplica a laboratorios. Quizzes tienen fecha límite estricta.

Sobre el uso de IA

Permitido: usar ChatGPT, Copilot, Claude como apoyo para aprender — entender conceptos, depurar código, explorar ideas.

No permitido: usar IA para resolver ejercicios evaluados ni para verificar respuestas antes de entregarlas.

Pruebas escritas (60%): en papel y lápiz, sin ningún recurso electrónico.
Incluyen problemas analíticos *y* de programación.

La capacidad de resolver problemas sin asistencia es una habilidad que este curso busca desarrollar.

Integridad académica

Todas las evaluaciones son **estrictamente individuales**.

Criterio orientador: la colaboración es aceptable si podría realizarse por teléfono (discutir ideas). No si requiere compartir documentos o pantallas.

Fraude o intento de fraude: proceso disciplinario según el reglamento de la UCU.

Comunicación

- **Dudas generales:** foro en la plataforma del curso
- **Consultas específicas:** pablo.iturralde@ucu.edu.uy
- **Soporte técnico:** j.paroli@ucu.edu.uy, m.barrutia@ucu.edu.uy

Bibliografía principal

- **[MML]** Deisenroth, Faisal & Ong — *Mathematics for Machine Learning* (gratuito: mml-book.com)
- **[LAOML]** Aggarwal — *Linear Algebra and Optimization for ML* (Timbó)
- **[ILA]** Strang — *Introduction to Linear Algebra*, 6.ª ed.
- **[SC]** Stewart — *Cálculo de varias variables*, 8.ª ed.

Bibliografía complementaria

- **[NLA]** Trefethen & Bau — *Numerical Linear Algebra*
- **[MC]** Golub & Van Loan — *Matrix Computations*, 4.ª ed.
- **[SCI]** Heath — *Scientific Computing: An Introductory Survey*, 2.ª ed.
- **[BV]** Boyd & Vandenberghe — *Convex Optimization* (gratuito)
- **[NW]** Nocedal & Wright — *Numerical Optimization*, 2.ª ed.
- **[GBC]** Goodfellow, Bengio & Courville — *Deep Learning* (gratuito)
- **[BB]** Bishop & Bishop — *Deep Learning: Foundations and Concepts*

¡Lean el Syllabus!

Está publicado en la plataforma del curso. Contiene toda la información de logística, evaluación, políticas y bibliografía en detalle.

Lo que vimos hoy fue un resumen. El syllabus es el documento oficial.

Parte III

Matemática para Aprendizaje Automático

¿Qué sistemas de inteligencia artificial usan todos los días?

IA está en todos lados

Recomendaciones: Netflix, Spotify, YouTube

Asistentes: Siri, Alexa, ChatGPT

Redes sociales: el algoritmo del feed, filtros

Smartphones: reconocimiento facial, texto predictivo

Transporte: predicción de tráfico, Uber/Lyft

Finanzas: detección de fraude, credit scoring

Salud: imágenes médicas, diagnóstico asistido

Ciencia: descubrimiento de fármacos, astronomía

Todo eso es aprendizaje automático

Detrás de cada uno de esos sistemas hay un modelo que **aprendió de datos**.

> *"Se dice que un programa aprende de la experiencia E con respecto a una tarea T y medida de rendimiento P, si su rendimiento en T, medido por P, mejora con la experiencia E."*
> *— Tom Mitchell (1997)*

Tres componentes mínimos:

- **Experiencia** = datos
- **Tarea** = predicción o decisión
- **Rendimiento** = medida de éxito

Programación tradicional

El programador escribe **reglas explícitas** que transforman los datos en resultados.

Datos + **Reglas** → Resultados

Ejemplo: si la temperatura > 30°C, encender el aire acondicionado.

Funciona bien cuando las reglas son claras y el problema es bien definido.

¿Pero qué pasa cuando no podemos escribir las reglas?

El giro del aprendizaje automático

En ML le damos al algoritmo los datos **y** las respuestas correctas, y él *descubre* las reglas.

Datos + **Resultados** → Reglas

Ejemplo: dado un historial de temperaturas y acciones, el algoritmo aprende cuándo encender el aire.

Las reglas ya no las escribe el programador. Las **aprende** el modelo.

Tres grandes familias de ML

Supervisado

Aprende de ejemplos **etiquetados**

Clasificación, regresión

No supervisado

Descubre **estructura** en los datos

Clustering, reducción de dimensionalidad

Por refuerzo

Aprende por **prueba y error** interactuando con un entorno

Control, juegos, decisión secuencial

Aprendizaje supervisado

El modelo recibe pares **(entrada, respuesta correcta)** y aprende a predecir la respuesta para entradas nuevas.

Ejemplos que ya conocen:

- **Reconocimiento de imágenes** — Google Photos identifica caras y objetos. Nuestro ejemplo gato/perro es exactamente esto.
- **Filtro de spam** — Gmail clasifica cada email como spam o no spam, entrenado con millones de emails etiquetados por usuarios.
- **Traducción automática** — DeepL y Google Translate aprenden de millones de pares de oraciones traducidas.
- **Pre-entrenamiento de LLMs** — GPT y Claude aprenden a predecir la siguiente palabra en un texto. La "etiqueta" es la palabra que efectivamente sigue.

Aprendizaje no supervisado

No hay etiquetas. El modelo busca **patrones y agrupamientos** en los datos por sí solo.

Ejemplos que ya conocen:

- **Spotify Discover Weekly / Netflix** — Agrupan usuarios con gustos similares (collaborative filtering) sin que nadie etiquete "este usuario es parecido a este otro". La estructura emerge de los datos de escucha/visualización.
- **Detección de anomalías** — Sistemas bancarios modelan el comportamiento "normal" de cada cliente. Una transacción que se aleja del patrón dispara una alerta, sin necesidad de ejemplos etiquetados de fraude.
- **PCA** — Reducir la dimensionalidad de datos complejos conservando la información esencial. Lo vamos a ver en este curso.

Aprendizaje por refuerzo

Un **agente** toma acciones en un entorno y recibe **recompensas o penalizaciones**. Aprende por prueba y error a maximizar la recompensa acumulada.

Ejemplos que ya conocen:

- **AlphaGo** (DeepMind) — Aprendió a jugar Go mejor que cualquier humano jugando millones de partidas contra sí mismo y recibiendo como señal únicamente si ganó o perdió.
- **Robótica** — Robots de Amazon aprenden a agarrar y mover objetos en almacenes, ajustando su estrategia con cada intento exitoso o fallido.
- **RLHF en ChatGPT y Claude** — Después del pre-entrenamiento (supervisado), los LLMs modernos se afinan con *Reinforcement Learning from Human Feedback*: humanos rankean respuestas, y el modelo aprende a generar las que los humanos prefieren.

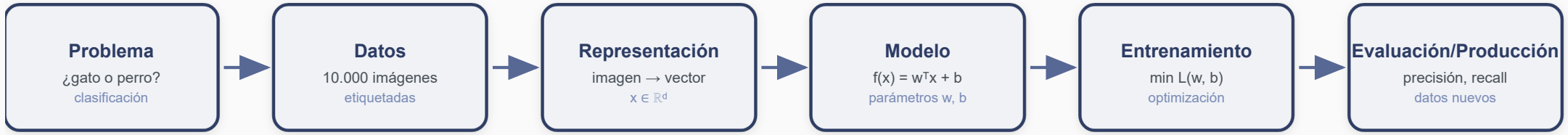
Volvamos a gatos y perros

Ahora podemos ser más precisos. Tenemos un problema de **clasificación supervisada**:

- **Experiencia E**: 10.000 imágenes etiquetadas (gato / perro)
- **Tarea T**: clasificar una imagen nueva
- **Rendimiento P**: proporción de aciertos (precisión)

¿Cómo pasamos de píxeles a una decisión?

El pipeline de ML



La matemática detrás del clasificador

Representación

Cada imagen de 64×64 píxeles → vector en $\mathbb{R}^{4096}$

El conjunto de datos es una **matriz** de 10.000 × 4.096

Modelo (ejemplo)

Clasificador lineal: $\hat{y} = f(x) = w^T x + b$

El mapa del curso

U1: Álgebra Lineal

Vectores, matrices, bases
Proyecciones, ortogonalidad
Autovalores, diagonalización
Costo computacional

5 semanas

Aplicación: PCA

U2: Sistemas Lineales

LU, Cholesky, QR
Mínimos cuadrados
SVD, pseudoinversa
Regularización

4 semanas

Aplicación: regresión, calibración

U3: Cálculo Multivariado

Gradientes, hessianos
Regla de la cadena
Backpropagation
Integración

4 semanas

Aplicación: redes neuronales

U4: Optimización

Convexidad
Descenso del gradiente
Multiplicadores de Lagrange
Restricciones

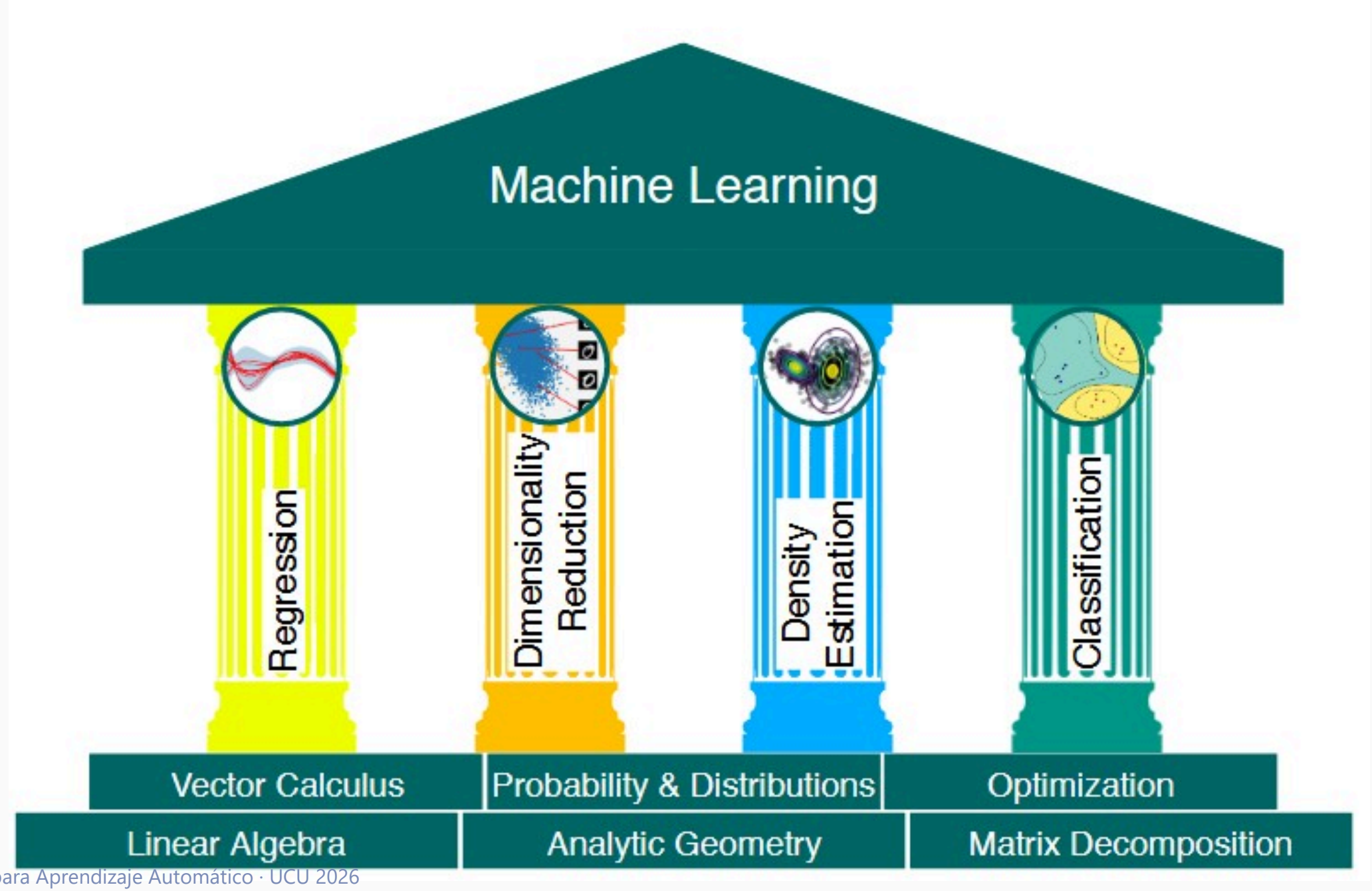
3 semanas

Aplicación: entrenamiento de modelos

Sem 1 · Sem 5 · Sem 9 · Sem 13 · Sem 16

La matemática no es un accesorio *al costado* de ML.

La matemática es la *base* de ML.


Machine Learning
Regression
Dimensionality Reduction
Density Estimation
Classification
Vector Calculus
Probability & Distributions
Optimization
Linear Algebra
Analytic Geometry
Matrix Decomposition

Qué sigue

Próxima clase:

Repaso operativo de álgebra lineal — notación, vectores, matrices, representación de datos como objetos algebraicos.

Para la próxima:

Leer MML cap. 2.1–2.7

¿Preguntas?

pablo.iturralde@ucu.edu.uy